

Motor de recomendación semántica multiperfil

Pipeline completo, estrategias elegidas y resultados (versión 1.4)

Documento de trabajo para el equipo del proyecto

26 de septiembre de 2026

Resumen

El motor recomienda películas a un perfil cinéfilo a partir de reseñas de Letterboxd. Las reseñas se vectorizan con un modelo de lenguaje; un *modelo de reglas en dos etapas* predice, desde esas reseñas, cuánto cumple cada película cada afinidad y cada filtro del perfil, y luego calcula la nota global con la forma de la regla global del perfil. Se entrena contra una Verdad Base de 110 películas evaluadas por Gemini. Con el modelo de embeddings **multilingual-e5-base** y las *frases de reseña* del perfil, el orden que produce el modelo se parece al de Gemini con ρ de Spearman = 0.674 en validación cruzada (0.55 con la regresión lineal de las versiones anteriores y 0 al azar), y la nota se equivoca en 0.58 puntos en promedio (0.81 prediciendo siempre el promedio). El documento explica cada paso con la estrategia elegida y la descartada, sigue una película de principio a fin (*Manhattan*), resume cómo ejecutar el sistema y los resultados, y propone los pasos siguientes.

Índice

1. El problema y los datos	3
2. El pipeline	4
2.1. Filtrado de reseñas (opción 2 , <code>src/db/filtered/filter.py</code>)	4
2.2. Chunks y embeddings de reseñas (opción 4 , <code>src/procesing_reviews.py</code>)	5
2.3. Embeddings del perfil (opción 3 , <code>src/procesing_profiles.py</code>)	5
2.4. Resumen de cada película (<i>pooling</i> , <code>src/pooling.py</code>)	5
2.5. Rasgos de frases de reseña (<code>src/pooling.py</code>)	5
2.6. Modelo de reglas en dos etapas (opción 7)	6
2.7. Recomendación (opción 8 , <code>src/recommend.py</code>)	7
2.8. Evaluación (opción 9 , <code>src/analisis.py</code> , <code>src/metricas.py</code>)	7
3. Ejemplo guiado: <i>Manhattan</i> (Woody Allen, 1979)	7
4. Cómo ejecutarlo	9
4.1. Instalación y datos	9
4.2. Menú principal	10
4.3. Variables de entorno	10
4.4. GitHub Actions	10
4.5. Archivos del análisis	10
5. Resultados	10

5.1. Modelo final y comparación de modelos de embeddings	11
5.2. Dónde se pierde precisión: la etapa 1	11
5.3. Pooling y cantidad de reseñas	12
5.4. Frases de reseña	13
5.5. ¿Cuánto ayudaría tener más películas evaluadas?	14
5.6. Recomendaciones	15
6. Observaciones	15
7. Pasos propuestos	16
A. Resumen de decisiones	16

1. El problema y los datos

Objetivo. Dado un perfil cinéfilo, ordenar películas que la persona no ha evaluado según cuánto le gustarían. Para recomendar importa más *el orden* que la nota exacta, por eso la métrica principal es la correlación de Spearman (ρ) entre el orden del modelo y el orden de la Verdad Base (sección 2.8).

El perfil (`src/db/profiles/perfiles.ejemplo.json`) describe el gusto en lenguaje natural y es genérico: puede tener cualquier cantidad de criterios.

- **Afinidades** (5): lo que la persona valora, p. ej. *Contemplación Inmersiva* o *Ternura y Empatía Radical*. Cada una tiene nombre, descripción y encabezado de planilla.
- **Filtros restrictivos** (5): lo que le molesta, p. ej. *Insoportabilidad Prolongada* (convivir con personajes arrogantes o crueles). Cada uno tiene descripción, severidad, las afinidades que *corrompe* y una **excepción** (cuándo el filtro no aplica). La cadena es **afinidad** $\leftarrow$ **filtro** $\leftarrow$ **excepción**: el filtro resta, salvo que su excepción lo neutralice.
- **Regla global**: cuatro puntos en texto que explican cómo combinar todo en una nota 0–10.
- **Frases de reseña** (opcional, nuevo en 1.2): frases como las escribiría alguien en Letterboxd, p. ej. “*Oscar bait designed to make you cry*” para *Control Emocional Artificial*.

Las reseñas. Un *scraper* guarda reseñas de Letterboxd en una base SQLite (`reviews.db`, 146 163 reseñas). Se publica solo como `reviews.zip` en el *release* en borrador `datos-resenas` del repositorio, visible únicamente para quienes tienen permiso de escritura: el repositorio es público y las reseñas no se deben publicar. Tras el filtrado (sección 2.1) quedan 14 864 reseñas de 150 películas, que se dividen en 55 769 fragmentos (*chunks*).

La Verdad Base. Gemini evalúa cada película con las instrucciones que genera la **opción 6** (`src/loss/instrucciones.py`): el perfil completo en texto, la regla global y el formato de salida. Para cada película entrega un puntaje 0–10 por afinidad y por filtro, si aplica la excepción de cada filtro (0/1) y la nota global. Hay 110 películas evaluadas (`dataset_maestro.csv` $\rightarrow$ `matriz_perdida.csv`); 101 tienen reseñas y sirven para entrenar, y 57 películas con reseñas aún no tienen nota: son las candidatas a recomendar.

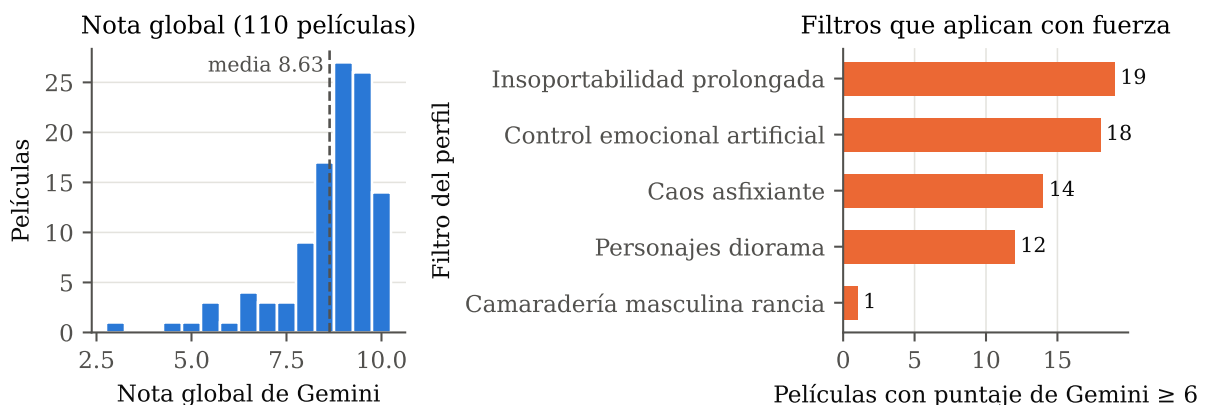


Figura 1: La Verdad Base es una lista de películas preferidas: la nota media es 8.63 y 93 de 110 tienen 8 o más (izquierda). Los filtros casi nunca aplican con fuerza; *Camaradería Masculina Rancia* tiene una sola película con puntaje ≥ 6 (derecha).

2. El pipeline

La figura 2 resume el flujo y la opción del menú (`python src/main.py`) que ejecuta cada paso. Las secciones siguientes explican cada paso: qué hace, la estrategia elegida, la variante básica o alternativa descartada y la evidencia para decidir. Todas las comparaciones se hicieron con validación cruzada repetida (sección 2.8).

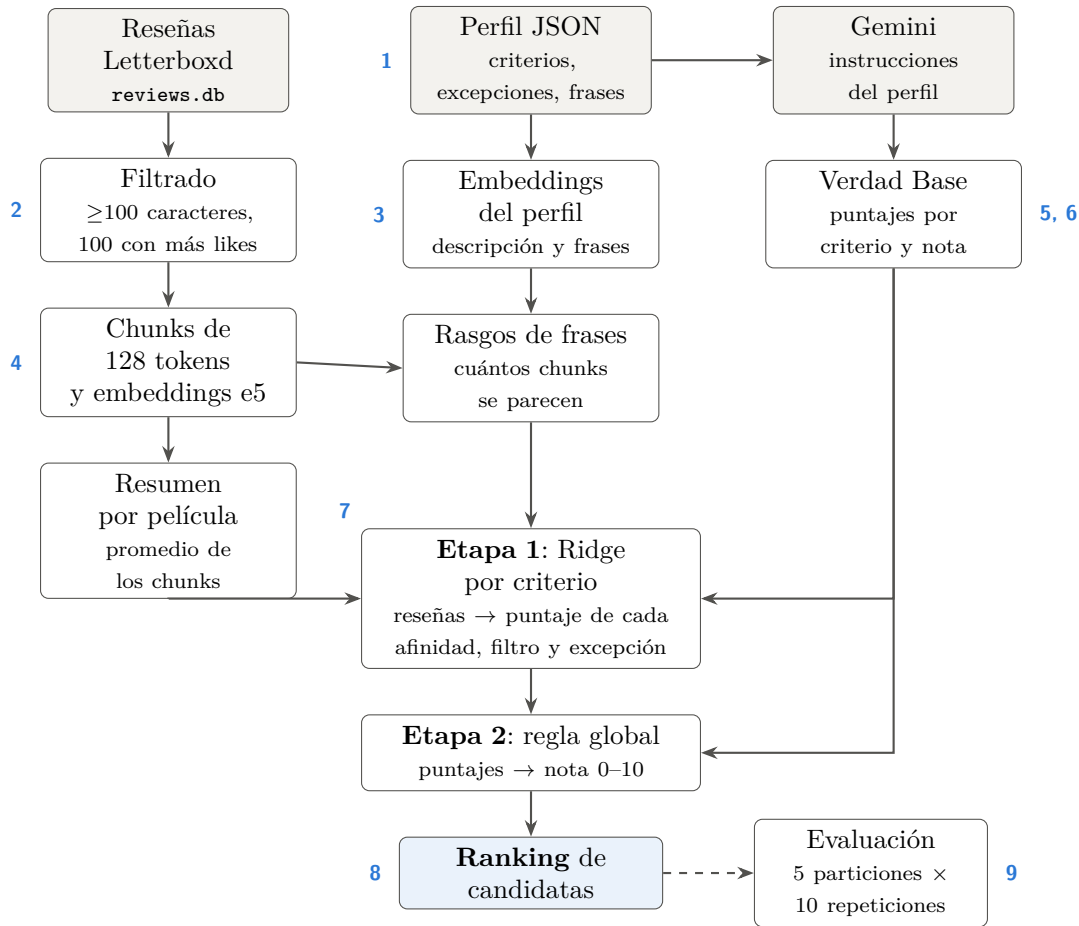


Figura 2: Pipeline. Los números azules son las opciones del menú; la **opción 9** ejecuta de la 2 a la 8 sin preguntas y agrega las comparaciones y el análisis.

2.1. Filtrado de reseñas (opción 2, `src/db/filtered/filter.py`)

Lee la base SQLite con una consulta con funciones de ventana: por cada película toma las reseñas con más *likes*, descartando las de menos de 100 caracteres (suelen ser chistes de una línea que no describen la película), y solo películas con al menos 20 reseñas. Limpia caracteres especiales y deja `film_id` (el *slug* de Letterboxd), título, director, idioma, *likes* y texto. Si el resultado es idéntico al último filtrado no crea otro archivo, para no duplicar embeddings después.

Elegida	hasta 100 reseñas por película, ordenadas por <i>likes</i> (<code>MAX_RESENIAS_POR_PELICULA</code>).
Descartadas	200 reseñas: ordena igual o peor y duplica el tiempo de generar embeddings (con MiniLM, 100 ordenó mejor que 200 en las 10 repeticiones, $\Delta\rho = +0.021$). 50 reseñas: peor con los tres modelos (fig. 5).

2.2. Chunks y embeddings de reseñas (opción 4, src/procesing_reviews.py)

Cada reseña se divide con `semchunk` en fragmentos de hasta 128 tokens y cada fragmento se convierte en un vector (*embedding*) con un modelo multilingüe de `sentence-transformers`. Los vectores se guardan en ChromaDB (colección `resenias`); cada modelo en su propia carpeta (`src/db/embeddings/chroma_<modelo>/`) porque sus dimensiones no se pueden mezclar. El identificador de cada chunk incluye película, lote, reseña y número de chunk, así que reprocesar un lote no duplica.

Elegida	<code>intfloat/multilingual-e5-base</code> (768 dimensiones), con el prefijo "query: " que exige la familia e5. Por defecto desde la versión 1.3.
Básica (descartada)	<code>paraphrase-multilingual-MiniLM-L12-v2</code> (384 dimensiones), la de las versiones anteriores: $\rho = 0.605$ contra 0.674 con e5 (tabla 3). Sigue disponible con la variable <code>MODELO_EMBEDDINGS</code> .
Alternativa	<code>paraphrase-multilingual-mpnet-base-v2</code> : mejora las afinidades pero no los filtros ($\rho = 0.651$).
Costo	e5 y mpnet tardan unas 3 veces más por chunk: 1 h 40 min en GitHub Actions para 55 769 chunks, sin GPU.

2.3. Embeddings del perfil (opción 3, src/procesing_profiles.py)

Cada afinidad y cada filtro se vectoriza con el mismo modelo, junto a un documento por excepción y uno por cada frase de reseña (colección `perfiles`).

Elegida	vectorizar solo nombre y descripción de cada criterio.
Descartada	el texto completo del criterio (con sus afinidades corrompidas y su excepción): acercaba el vector del filtro a los conceptos que se oponen a él. La similitud entre <i>Insoportabilidad</i> y <i>Ternura</i> bajó de 0.68 a 0.50 al quitar ese texto.

2.4. Resumen de cada película (pooling, src/pooling.py)

Una película tiene cientos de chunks; la etapa 1 necesita un solo vector por película.

Elegida	promedio de los embeddings de todos sus chunks (<code>POOLING_RESENIAS=media</code>).
Descartadas	percentil 50, 75, 90 o 95 de cada dimensión del embedding, y promedio más percentiles de la similitud de los chunks con cada término del perfil. Con e5 ninguna ordena mejor que el promedio en 10 repeticiones (la mejor, $\Delta\rho = +0.001$; el percentil 75, -0.021 , peor en las 10). Con una sola partición el percentil 90 parecía ganar (0.584 contra 0.606 de MAE), pero era ruido (fig. 5). Se pueden elegir con <code>POOLING_RESENIAS=pXX</code> .

2.5. Rasgos de frases de reseña (src/pooling.py)

La descripción de un filtro está escrita como la escribiría un crítico (“control emocional artificial”); las reseñas dicen “Oscar bait”. Un experimento previo (`src/experimentos/sondas_filtros.py`) midió, para cada texto, qué fracción de los chunks de cada película se parece mucho a él y la correlacionó con el puntaje de Gemini del filtro (tabla 6): las frases con el vocabulario de las reseñas detectan mejor cuatro de los cinco filtros.

Para cada criterio con `frases_resenia`, y para cada texto t (su descripción y cada frase), se calcula:

- la **fracción de chunks** de la película con similitud coseno con t mayor que el percentil 95 de esa similitud en una muestra de 20 000 chunks de todas las películas (el umbral no usa notas), y
- el **percentil 90** de la similitud de los chunks de la película con t .

Con 3–4 frases por filtro son 8–10 rasgos. El umbral de cada texto se guarda con el modelo para medir igual a las películas que se recomiendan después.

Elegida	ponderar: los rasgos se agregan al embedding y, tras estandarizar, se multiplican por $\sqrt{768/k}$ (k = cantidad de rasgos), para que pesen en conjunto lo mismo que las 768 dimensiones del embedding. Con e5 mejora el orden en las 10 repeticiones ($\Delta\rho = +0.008$) y baja el MAE.
Descartadas	concatenarlos sin ponderar (Ridge los diluye: $+0.002$); usar solo las frases, sin el embedding ($+0.011$ en ρ pero sube el MAE y con mpnet no se repite); y los controles con solo la descripción ($+0.000$ y $+0.005$), que muestran que la ganancia viene de las frases y no solo de medir “cuántas reseñas hablan de”.

2.6. Modelo de reglas en dos etapas (opción 7)

Implementado en `src/modelo_reglas.py` y entrenado por `src/scoring.py`.

Etapla 1: de las reseñas a los puntajes. Para cada criterio c (afinidad, filtro o excepción) una regresión Ridge, con estandarización y la regularización elegida por validación cruzada interna ($\alpha \in [10^{-1}, 10^4]$), predice el puntaje de Gemini a partir del embedding promedio de la película y, si el criterio tiene frases, de sus rasgos. Las predicciones se recortan a $[0, 10]$ (a $[0, 1]$ para las excepciones). Son 11 579 coeficientes regularizados en total (768 + rasgos por criterio).

Etapla 2: de los puntajes a la nota. La regla global del perfil se traduce a una fórmula con parámetros que se aprenden de la Verdad Base:

$$\text{nota} = b + \sum_{j \in \text{afinidades}} w_j A_j - \sum_{k \in \text{filtros}} d_k \sigma(1.5 (F_k - u_k)) (1 - E_k), \quad (1)$$

donde A_j , F_k y E_k son los puntajes de afinidades, filtros y excepciones, $w_j \in [0, 2]$ el peso de cada afinidad, u_k el umbral desde el que el filtro k “se activa”, d_k cuánto resta como máximo, y σ la sigmoide, que hace suave el umbral para poder optimizarlo. La excepción ($E_k = 1$) anula el descuento. Se ajusta con L-BFGS-B minimizando el error cuadrático, con tres inicios y una penalización L2 pequeña (10^{-3}) sobre los descuentos. Tiene $1 + n_A + 2n_F$ parámetros: 16 para este perfil, y la misma forma sirve para cualquier perfil. La etapa 2 se entrena con las 110 películas de la Verdad Base, porque no necesita reseñas.

La regla aprendida con e5 es:

- Nota base = $4.19 + 0.08 \cdot \text{Resistencia Femenina} + 0.12 \cdot \text{Contemplación Inmersiva} + 0.16 \cdot \text{Ternura y Empatía Radical} + 0.09 \cdot \text{Humanismo Social} + 0.19 \cdot \text{Vanguardia y Simbolismo}$
- Si Insoportabilidad Prolongada supera 5.4 y su excepción no aplica: resta hasta 1.66 puntos.
- Si Control Emocional Artificial supera 8.8 y su excepción no aplica: resta hasta 2.98 puntos.
- Si Personajes Diorama supera 4.0 y su excepción no aplica: resta hasta 0.45 puntos.
- Si Caos Asfixiante supera 0.9 y su excepción no aplica: resta hasta 0.25 puntos.

Elegida Básica (descartada)	modelo de reglas de dos etapas con la regla simple (1). la regresión lineal de las versiones anteriores: similitud coseno entre el embedding de la película y el de cada término del perfil, más un producto por relación (afinidad $\times$ filtro, filtro $\times$ excepción), 21 variables. Ordena peor ($\rho = 0.546$ contra 0.674 con e5) y con MiniLM su error era mayor que predecir el promedio (MAE 0.828 contra 0.812). La opción 9 la sigue calculando como comparación.
Alternativas de etapa 2	regla con <i>pisos</i> (una afinidad alta asegura una nota mínima) y <i>corrupción</i> (el filtro reduce las afinidades que corrompe): MAE 0.293 contra 0.294 con casi el doble de parámetros (31); quedan como opciones desactivadas. También se probaron regresión lineal (0.333), GAM, <i>gradient boosting</i> con y sin restricciones monótonas y <i>random forest</i> (peores con 110 películas) y la regla corregida con <i>boosting</i> (0.280 , apenas mejor y mucho menos legible).

2.7. Recomendación (opción 8, src/recommend.py)

Carga el modelo guardado por la **opción 7** (src/loss/modelo_reglas_<modelo>.joblib, con el pooling, las frases y sus umbrales) y ordena todas las películas con reseñas por la nota predicha. Las que ya tienen nota de Gemini aparecen para comparar.

2.8. Evaluación (opción 9, src/analisis.py, src/metricas.py)

- **Validación cruzada:** cada película se predice con un modelo que no la vio. 5 particiones, repetidas con 10 semillas distintas; se informa el promedio y la desviación. Con unas 100 películas, el ρ de una sola partición varía ± 0.05 , tanto como las diferencias que se quieren medir.
- **Comparaciones pareadas:** cada variante se compara contra una referencia *en las mismas particiones* ($\Delta\rho$), que varía mucho menos, y se cuenta en cuántas de las 10 repeticiones la supera.
- **Métricas:** ρ de Spearman (principal: el orden completo); NDCG@10 (si las 10 primeras según el modelo son las mejores según Gemini, en ese orden, con ganancia $y - \text{mín } y$); precisión@10 (fracción de las 10 primeras que están entre las 10 mejores, contando empates en el corte), y MAE de la nota. La línea base (predecir el promedio) se informa con los valores esperados de un orden al azar.

Elegida	ρ de Spearman como métrica principal, promedio de 10 repeticiones.
Descartada	MAE como métrica principal y una sola partición: favorecía variantes que no ordenan mejor (las frases de reseña casi no cambian el MAE pero sí el orden) y hacía que variantes iguales parecieran distintas.

3. Ejemplo guiado: *Manhattan* (Woody Allen, 1979)

Manhattan es una de las pocas películas de la lista con nota baja (Gemini: 5.0) y la razón es un filtro: la película obliga a convivir con un protagonista cuya relación con una adolescente de 17 años y cuyo aire de superioridad intelectual encajan en *Insoportabilidad Prolongada* (Gemini: 8 de 10, sin excepción). Además Gemini le da puntajes bajos en tres afinidades. Es un caso difícil y por eso instructivo: muestra cada paso y también dónde falla el modelo. Para que el ejemplo sea honesto, el modelo de esta sección se entrenó **con todas las demás películas y sin Manhattan**, como si fuera una candidata nueva.

Paso 1: reseñas. El filtrado toma sus 100 reseñas con más *likes* y al menos 100 caracteres, en inglés, portugués, persa, árabe y otros idiomas. Entre ellas hay críticas al personaje, que mencionan su edad y lo tratan de *pretentious* y *creep*, pero son una minoría frente a las que elogian la fotografía en blanco y negro y la música de Gershwin.

Paso 2: chunks y embeddings. Las reseñas se dividen en 287 chunks de hasta 128 tokens y cada uno se convierte en un vector de 768 dimensiones con e5. El promedio de esos vectores es la representación de la película. En el promedio, las críticas de la minoría se diluyen: para eso están los rasgos de frases.

Paso 3: rasgos de frases. Para *Insoportabilidad Prolongada*, la tabla 1 muestra qué fracción de los chunks de Manhattan supera el umbral de similitud con cada texto. Con la frase “*I couldn’t stand the protagonist, he is a pretentious narcissist and a creep*”, el 11.5 % de sus chunks la supera, contra 4.2 % en la película mediana: Manhattan está en el percentil 95 de las 101 películas. Con la descripción del perfil, escrita en otro registro, queda en el percentil 26. Las frases sí encuentran el problema.

Texto	Manhattan	Mediana	Percentil
<i>descripción</i>	2.1 %	3.8 %	26
<i>the main character is insufferable, arrogant and smug</i>	7.3 %	4.6 %	85
<i>I couldn’t stand the protagonist, he is a pretentious narcissist and a creep</i>	11.5 %	4.2 %	95
<i>an unlikely, annoying character you have to spend the whole movie with</i>	5.6 %	4.7 %	65
<i>un protagonista insoportable, arrogante y pedante</i>	6.3 %	4.1 %	67

Tabla 1: Rasgos de frases de *Insoportabilidad Prolongada* para Manhattan: fracción de sus chunks sobre el umbral de cada texto, la misma fracción en la película mediana y el percentil de Manhattan entre las 101 películas.

Pasos 4 y 5: etapa 1 y regla global. La etapa 1 predice los 14 puntajes desde el embedding y los rasgos, y la regla (1) los combina (tabla 2). La nota base es $b + \sum_j w_j A_j = 8.31$ y los filtros restan 0.03 en total: la nota predicha queda en **8.28**, lejos del 5.0 de Gemini.

Criterio	Gemini	Etapa 1 (predicho)	Parámetros de la regla	Aporte
Sesgo				3.74
Resistencia Femenina	2.0	7.0	$w = 0.10$	+0.69
Contemplación Inmersiva	6.0	7.0	$w = 0.11$	+0.74
Ternura y Empatía Radical	2.0	7.2	$w = 0.15$	+1.09
Humanismo Social	3.0	6.2	$w = 0.13$	+0.79
Vanguardia y Simbolismo	6.0	6.3	$w = 0.20$	+1.26
Camaradería Masculina Rancia	9.0 (exc. 1)	0.3 (exc. 0.11)	$u = 4.9, d = 0.05, \text{act. } 0.00$	0.00
Insoportabilidad Prolongada	8.0 (exc. 0)	3.0 (exc. 0.36)	$u = 5.6, d = 1.68, \text{act. } 0.01$	−0.02
Control Emocional Artificial	3.0 (exc. 0)	2.9 (exc. 0.00)	$u = 8.8, d = 2.16, \text{act. } 0.00$	0.00
Personajes Diorama	4.0 (exc. 0)	3.3 (exc. 0.01)	$u = 7.1, d = 0.65, \text{act. } 0.00$	0.00
Caos Asfixiante	0.0 (exc. 0)	0.0 (exc. 0.00)	$u = 2.2, d = 0.24, \text{act. } 0.04$	−0.01
Nota predicha (Gemini: 5.0)				8.28

Tabla 2: Cálculo de la nota de Manhattan con el modelo entrenado sin ella. Para los filtros, “act.” es $\sigma(1.5(F_k - u_k))(1 - E_k)$, la fracción del descuento d_k que se aplica; “exc.” es la excepción.

Dónde falla. La etapa 2 hace lo que dice la regla, pero recibe puntajes equivocados de la etapa 1:

- **Afinidades:** la etapa 1 predice cerca de 7 para todas, el valor típico de la lista, mientras Gemini da 2 o 3 en tres de ellas. Con 101 películas casi todas bien evaluadas, Ridge aprende a quedarse cerca del promedio.
- **El filtro:** aunque el rasgo de la frase está en el percentil 95, la etapa 1 predice 3.0 para *Insoportabilidad*, bajo su umbral de 5.6, y el descuento no se activa. Hay 19 películas con ese filtro alto: pocas para que Ridge le dé tanto peso al rasgo.
- **Camaradería Masculina Rancia:** Gemini le da 9, pero con su excepción aplicada; Manhattan es la *única* película de la lista con ese filtro alto, así que al sacarla del entrenamiento no queda ningún ejemplo.

En el ranking. En la validación cruzada de 10 repeticiones, Manhattan queda en el puesto 84 de 101 (nota predicha 8.32; en Gemini es la 100). El caso contrario es *Marty Supreme* (Gemini 5.5, puesto 98): el modelo la deja última (101 de 101, nota 6.91). *Amélie* (4.5, la peor nota con reseñas) queda en el puesto 81: sus reseñas describen lo dulce y excéntrico como virtud, mientras el perfil lo considera *Control Emocional Artificial*. Los errores grandes se concentran en películas cuyo defecto, según el perfil, las reseñas no describen como defecto, o describen en pocas reseñas. Más películas con notas bajas en la Verdad Base atacarían ambos problemas (sección 7).

4. Cómo ejecutarlo

4.1. Instalación y datos

```
pip install -r requirements.txt
# perfil de ejemplo:
cp src/db/profiles/perfiles.ejemplo.json src/db/profiles/perfiles.json
python src/main.py
```

1. Descargar del *release* en borrador **datos-resenas** el archivo **reviews.zip** y descomprimirlo en **src/db/raw/** (queda **reviews.db**).
2. **Embeddings:** generarlos en CPU con e5 (opciones 2 a 4) tarda varias horas. El mismo *release* tiene la base ya generada, **embeddings-multilingual-e5-base-100-<huella>.zip**: descomprimirla en la raíz del repositorio. Crea **src/db/embeddings/chroma_multilingual-e5-base/** y el CSV filtrado correspondiente; las opciones 2 y 4 reconocen que ya están procesados.
3. Ejecutar la **opción 9** (todo el pipeline y el análisis), o las opciones 3, 5, 7 y 8 por separado.

4.2. Menú principal

Opción	Qué hace
1	Crear o editar perfiles: criterios, excepciones, encabezados, regla global y frases de reseña.
2	Filtrar las reseñas crudas (<code>src/db/raw/*.db</code> o <code>*.csv</code>).
3	Embeddings del perfil (descripciones, excepciones y frases).
4	Embeddings de los lotes de reseñas pendientes.
5	Cargar la Verdad Base desde <code>dataset_maestro.csv</code> y las evaluaciones adicionales.
6	Listar películas con reseñas sin nota y generar la plantilla y las instrucciones para Gemini.
7	Entrenar el modelo de reglas; informa ρ , NDCG@10, precisión@10 y MAE en validación cruzada y las reglas aprendidas.
8	Ranking de recomendaciones con el modelo guardado.
9	Ejecutar de la 2 a la 8 sin preguntas y guardar el análisis en <code>analisis/<fecha>_<modelo>/</code> .
10	Salir.

4.3. Variables de entorno

Variable	Uso
MODELO_EMBEDDINGS	Modelo de embeddings (por defecto <code>intfloat/multilingual-e5-base</code>).
MAX_RESENIAS_POR_PELICULA	Reseñas por película (por defecto 100).
POOLING_RESENIAS	<code>media</code> (por defecto) o <code>pXX</code> . El modelo guardado recuerda cuál usó.

4.4. GitHub Actions

- `analisis.yml`: ejecuta la **opción 9** con MiniLM, mpnet y e5 en paralelo, reutiliza los embeddings del caché y publica los resultados en la rama `resultados-analisis` (solo números, nunca texto de reseñas). El job de e5 publica su base de embeddings en el *release* en borrador. Corre al cambiar el workflow, el perfil de ejemplo, `analisis.py`, `modelo_reglas.py` o `pooling.py`.
- `sondas.yml`: el experimento de frases sonda de la sección 2.5.
- `tests.yml`: en cada *push*, 30 pruebas unitarias con datos sintéticos (`python -m pytest tests`) y la revisión estática del código con `pyflakes`.

4.5. Archivos del análisis

`resumen.md` (todas las tablas), `peliculas.csv` (por película: similitudes, puntajes predichos, notas y errores), `ranking.csv`, `reglas.json`, `pooling.csv`, `frases_resenia.csv`, `pesos.csv` y `similitud_perfil.csv`. Este informe se regenera con `python docs/informe/figuras.py`, `python docs/informe/tablas.py` y `latexmk -pdf informe.tex` desde `docs/informe/`.

5. Resultados

Las tablas 3 a 8 y las figuras 5 y 6 vienen de la corrida del workflow de análisis del 26 de septiembre (commit `9669c1c`), publicada en la rama `resultados-analisis`. La curva de

aprendizaje, la dispersión y el ejemplo guiado se calcularon con el mismo código sobre la base de embeddings de e5 publicada en el *release*. Los datos agregados de cada tabla y figura están en [docs/informe/datos/](#).

5.1. Modelo final y comparación de modelos de embeddings

Embeddings	Modelo	ρ Spearman	NDCG@10	Precisión@10	MAE
MiniLM	Reglas + frases (op. 7 y 8)	0.605 ± 0.024	0.929	0.280	0.632
	Regresión lineal anterior	0.464	0.909	0.200	0.828
	Regresión lineal con Ridge	0.400	0.886	0.300	0.809
mpnet-base	Reglas + frases (op. 7 y 8)	0.651 ± 0.024	0.934	0.260	0.601
	Regresión lineal anterior	0.598	0.906	0.200	0.891
	Regresión lineal con Ridge	0.605	0.906	0.300	0.745
e5-base	Reglas + frases (op. 7 y 8)	0.674 ± 0.026	0.938	0.280	0.580
	Regresión lineal anterior	0.546	0.897	0.300	0.735
	Regresión lineal con Ridge	0.597	0.944	0.500	0.659
Orden al azar / predecir el promedio		0	0.780	0.129	0.812

Tabla 3: Validación cruzada (101 películas). El modelo de reglas es el promedio de 10 repeticiones $\pm$ su desviación; las regresiones lineales, una partición. En negrita, el modelo que usan las opciones 7 y 8.

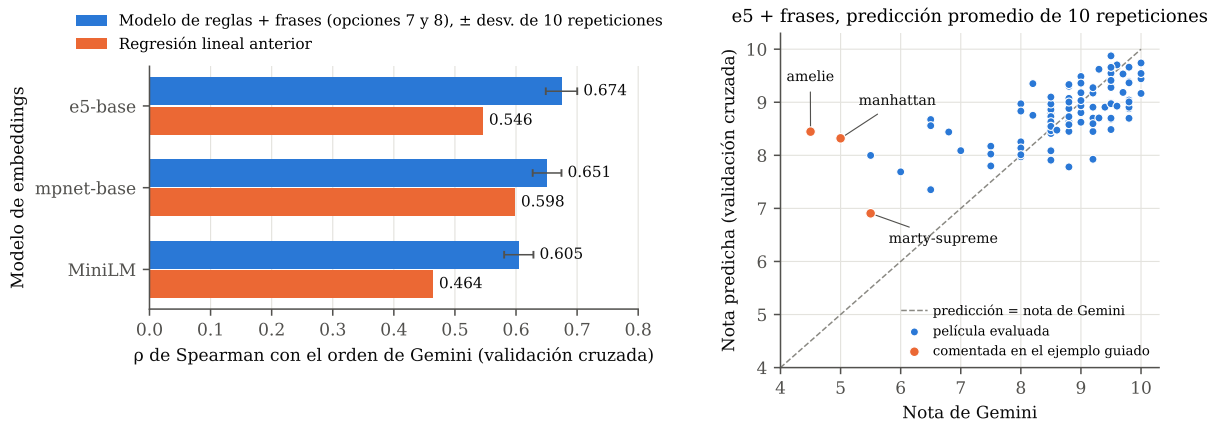


Figura 3: Izquierda: ρ de Spearman por modelo de embeddings (datos de la tabla 3). Derecha: nota de Gemini contra la nota predicha en validación cruzada con e5; en naranja, películas comentadas en el texto. El modelo comprime las notas hacia el promedio: sube las peores y baja las mejores, pero conserva buena parte del orden.

5.2. Dónde se pierde precisión: la etapa 1

Con los puntajes reales de Gemini, la etapa 2 explica el 88 % de la varianza de la nota global (tabla 4). Con los puntajes que predice la etapa 1 desde las reseñas, el modelo completo explica el 39 %. El cuello de botella es predecir los criterios desde las reseñas, sobre todo los filtros (fig. 4).

Entrada de la etapa 2	ρ Spearman	R^2	MAE
<i>Puntajes reales de Gemini (110 películas)</i>			
Regla con condiciones (16 parámetros)	0.904	0.882	0.317
Regresión lineal sobre los puntajes	0.898	0.881	0.325
Predecir el promedio	0.000	-0.013	0.911
Modelo completo (etapa 1 desde reseñas + regla), e5	0.674	0.394	0.580

Tabla 4: Etapa 2 por separado (validación cruzada sobre los puntajes de Gemini) y modelo completo.

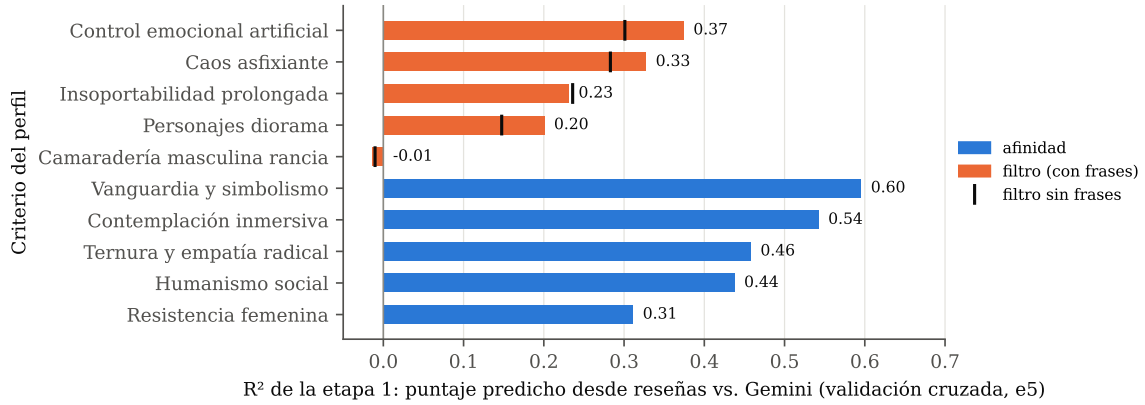


Figura 4: R^2 de la etapa 1 por criterio (e5). Las afinidades se predicen mejor que los filtros; las frases de reseña mejoran los filtros (barra) respecto del modelo sin frases (marca negra). *Camaradería Masculina Rancia* no se puede aprender: una sola película con puntaje alto.

5.3. Pooling y cantidad de reseñas

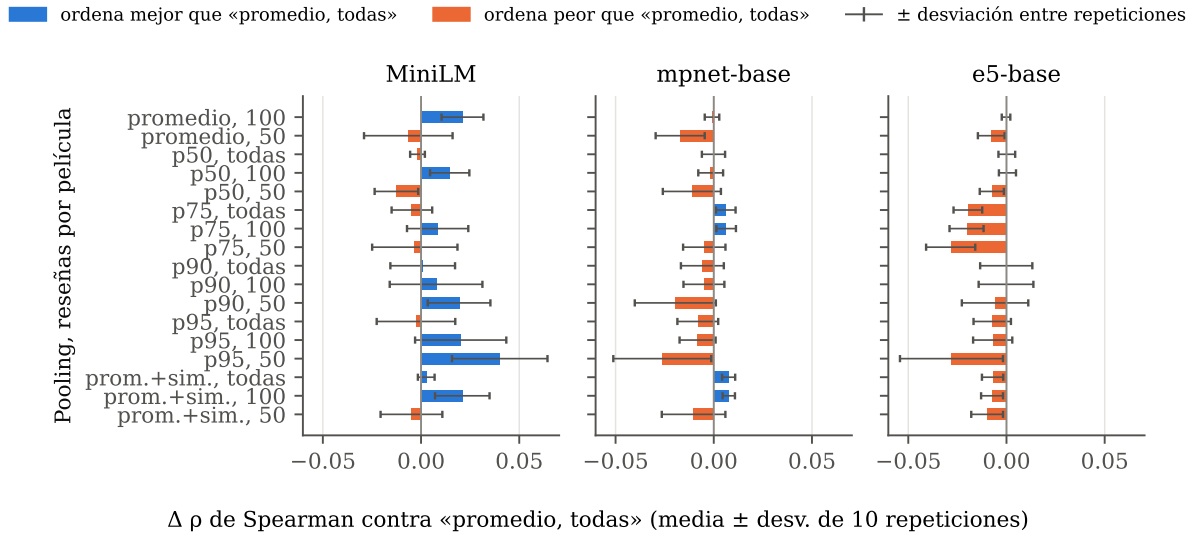


Figura 5: Diferencia de ρ de cada forma de resumir las reseñas contra el promedio de todas (media $\pm$ desviación de 10 repeticiones pareadas). Con mpnet y e5 “todas” son 100 reseñas. Ninguna variante mejora de forma consistente con los tres modelos; con e5, el promedio es el mejor.

Variante	MiniLM		mpnet-base		e5-base	
	$\Delta\rho$	mejora	$\Delta\rho$	mejora	$\Delta\rho$	mejora
promedio, todas (elegida)	$\rho = 0.596$ (ref.)		$\rho = 0.644$ (ref.)		$\rho = 0.667$ (ref.)	
promedio, 100	+0.021 $\pm$ 0.011	10/10	-0.001 $\pm$ 0.004	3/10	-0.000 $\pm$ 0.002	4/10
promedio, 50	-0.006 $\pm$ 0.023	5/10	-0.017 $\pm$ 0.012	1/10	-0.008 $\pm$ 0.007	1/10
percentil 50, todas	-0.002 $\pm$ 0.004	3/10	-0.000 $\pm$ 0.006	5/10	+0.000 $\pm$ 0.004	5/10
percentil 50, 100	+0.015 $\pm$ 0.010	9/10	-0.002 $\pm$ 0.006	4/10	+0.000 $\pm$ 0.004	6/10
percentil 50, 50	-0.013 $\pm$ 0.011	1/10	-0.011 $\pm$ 0.015	3/10	-0.007 $\pm$ 0.006	1/10
percentil 75, todas	-0.005 $\pm$ 0.010	4/10	+0.006 $\pm$ 0.005	9/10	-0.020 $\pm$ 0.007	0/10
percentil 75, 100	+0.008 $\pm$ 0.016	6/10	+0.006 $\pm$ 0.005	9/10	-0.020 $\pm$ 0.009	0/10
percentil 75, 50	-0.003 $\pm$ 0.022	5/10	-0.005 $\pm$ 0.011	3/10	-0.028 $\pm$ 0.013	0/10
percentil 90, todas	+0.001 $\pm$ 0.016	5/10	-0.006 $\pm$ 0.011	4/10	-0.000 $\pm$ 0.013	5/10
percentil 90, 100	+0.008 $\pm$ 0.024	7/10	-0.005 $\pm$ 0.010	5/10	-0.000 $\pm$ 0.014	5/10
percentil 90, 50	+0.019 $\pm$ 0.016	9/10	-0.020 $\pm$ 0.021	2/10	-0.006 $\pm$ 0.017	4/10
percentil 95, todas	-0.003 $\pm$ 0.020	5/10	-0.008 $\pm$ 0.010	3/10	-0.007 $\pm$ 0.010	3/10
percentil 95, 100	+0.020 $\pm$ 0.023	8/10	-0.008 $\pm$ 0.009	2/10	-0.007 $\pm$ 0.010	3/10
percentil 95, 50	+0.040 $\pm$ 0.024	9/10	-0.026 $\pm$ 0.025	2/10	-0.028 $\pm$ 0.026	2/10
promedio + sim., todas	+0.003 $\pm$ 0.004	7/10	+0.008 $\pm$ 0.003	10/10	-0.007 $\pm$ 0.005	0/10
promedio + sim., 100	+0.021 $\pm$ 0.014	10/10	+0.008 $\pm$ 0.003	10/10	-0.007 $\pm$ 0.006	1/10
promedio + sim., 50	-0.005 $\pm$ 0.016	3/10	-0.010 $\pm$ 0.016	2/10	-0.010 $\pm$ 0.008	1/10

Tabla 5: Datos de la figura 5: $\Delta\rho$ media $\pm$ desviación y repeticiones (de 10) en que la variante supera a la referencia. En negrita, mejoras en al menos 9 de 10.

5.4. Frases de reseña

Filtro	Descripción	Mejor frase	Léxico	Frase con mejor ρ
Camaradería Masculina Rancia	0.01	0.08	-0.03	<i>“humor machista y sexista, las mujeres son un chiste”</i>
Insoportabilidad Prolongada	0.39	0.53	0.55	<i>“I couldn’t stand the protagonist, he is a pretentious narcissist and a creep”</i>
Control Emocional Artificial	-0.04	0.40	0.16	<i>“Oscar bait designed to make you cry”</i>
Personajes Diorama	0.03	0.28	0.17	<i>“style over substance, the characters feel like dolls in a dollhouse”</i>
Caos Asfixiante	0.30	0.17	0.34	<i>“overstimulating, loud and relentless, too much happening at once”</i>

Tabla 6: Experimento de frases sonda (MiniLM, 101 películas): ρ de Spearman entre la fracción de chunks muy similares a cada texto y el puntaje de Gemini del filtro, y la correlación de un conteo de palabras clave (léxico). La descripción del perfil, escrita como la escribiría un crítico, detecta mal *Control* y *Diorama*.

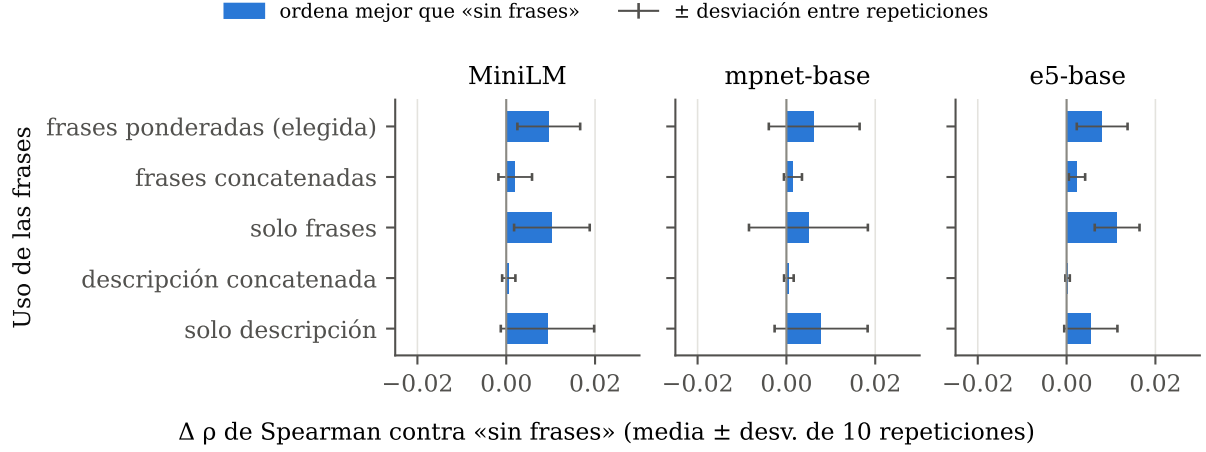


Figura 6: Diferencia de ρ de cada forma de usar las frases contra el modelo sin frases (10 repeticiones pareadas). Con e5, las frases ponderadas mejoran en las 10 repeticiones.

Variante	MiniLM		mpnet-base		e5-base	
	$\Delta\rho$	mejora	$\Delta\rho$	mejora	$\Delta\rho$	mejora
sin frases (referencia)	$\rho = 0.596$ (ref.)		$\rho = 0.644$ (ref.)		$\rho = 0.667$ (ref.)	
frases ponderadas (elegida)	$+0.010 \pm 0.007$	9/10	$+0.006 \pm 0.010$	7/10	$+0.008 \pm 0.006$	10/10
frases concatenadas	$+0.002 \pm 0.004$	7/10	$+0.001 \pm 0.002$	6/10	$+0.002 \pm 0.002$	10/10
solo frases	$+0.010 \pm 0.009$	10/10	$+0.005 \pm 0.013$	5/10	$+0.011 \pm 0.005$	10/10
descripción concat. (control)	$+0.001 \pm 0.001$	8/10	$+0.001 \pm 0.001$	8/10	$+0.000 \pm 0.000$	8/10
solo descripción (control)	$+0.009 \pm 0.011$	8/10	$+0.008 \pm 0.010$	7/10	$+0.005 \pm 0.006$	7/10

Tabla 7: Datos de la figura 6. En negrita, mejoras en al menos 9 de 10 repeticiones.

5.5. ¿Cuánto ayudaría tener más películas evaluadas?

La curva de aprendizaje (fig. 7) entrena con una fracción de las películas de cada partición y predice el resto. ρ sube de 0.45 con 20 películas a 0.60 con 40, 0.65 con 61 y 0.675 con 81: todavía sube, pero cada vez menos ($+0.026$ entre 61 y 81). Más películas parecidas a las actuales darían una mejora moderada; películas con notas bajas y con los filtros presentes deberían rendir más, porque aportan justo la variedad que falta (sección 7).

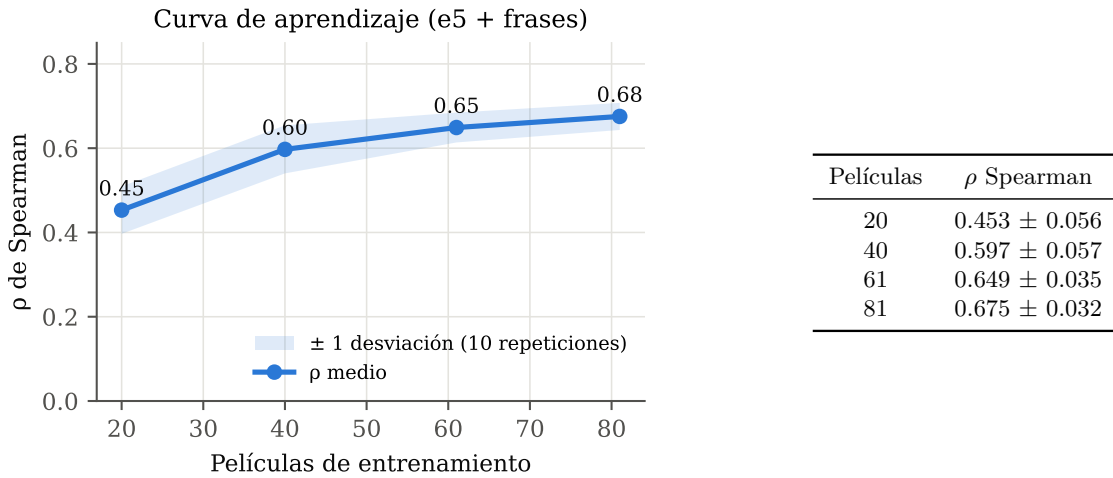


Figura 7: Curva de aprendizaje (e5 + frases, 5 particiones $\times$ 10 repeticiones): ρ medio $\pm$ desviación según la cantidad de películas de entrenamiento.

5.6. Recomendaciones

#	Película	Nota del modelo	Nota lineal anterior
1	taipei-suicide-story	9.75	10.30
2	under-the-skin	9.52	9.42
3	train-dreams	9.45	9.49
4	2046	9.43	8.53
5	the-passion-of-joan-of-arc	9.32	8.64
6	the-seventh-seal	9.30	10.17
7	sentimental-value	9.29	8.56
8	happy-together	9.23	9.15
9	chungking-express	9.18	8.50
10	neon-genesis-evangelion	9.12	8.97

Tabla 8: Las 10 mejores candidatas (sin nota de Gemini) según el modelo de reglas con e5, y la nota que les daba la regresión lineal anterior.

6. Observaciones

1. **La Verdad Base es una lista de películas preferidas.** Con media 8.63 y 93 de 110 películas con 8 o más, el modelo aprende sobre todo a distinguir “muy buena” de “excelente”. Los filtros casi nunca aplican (fig. 1), así que hay pocos ejemplos para aprender a detectarlos, y *Camaradería Masculina Rancia* tiene uno solo. El rango estrecho también limita el ρ alcanzable: con notas casi iguales, pequeñas diferencias cambian el orden.
2. **La Verdad Base es la lectura de Gemini, no la nota de la persona.** El modelo aprende a imitar cómo Gemini interpreta el perfil. Falta medir cuánto se parece eso al gusto real y cuán consistente es Gemini consigo mismo: si al reevaluar las mismas películas su propio ρ ronda 0.8, el 0.67 del modelo estaría cerca del techo.
3. **El cuello de botella es la etapa 1** (tabla 4): la regla explica bien la nota a partir de los puntajes, pero predecir los puntajes desde reseñas es difícil, en especial los filtros, que suelen estar en una minoría de reseñas y se diluyen en el promedio.
4. **El ruido de evaluación es del tamaño de las mejoras.** Una sola partición de la validación cruzada varía ± 0.05 en ρ ; varias conclusiones tempranas (el percentil 90, el ranking de modelos con una partición) se revirtieron al repetir. Toda decisión debe tomarse con comparaciones pareadas repetidas.
5. **La etapa 2 tiene óptimos locales parecidos:** diferencias de redondeo pueden llevar al optimizador a otra solución con casi el mismo error; afecta ρ en ~ 0.004 .
6. **Las frases se eligieron después de leer reseñas** de algunas películas de la Verdad Base, así que su ganancia puede ser algo optimista.
7. **Los chunks en otros idiomas** (13 % de las reseñas) son el 35 % de los más similares a las frases (*hubness* del modelo multilingüe), pero usar solo inglés no cambió las correlaciones (± 0.04).
8. **Privacidad:** el repositorio es público. Las reseñas y las bases de embeddings (que contienen fragmentos de reseñas) solo van al *release* en borrador; los resultados publicados son agregados e identificadores.

7. Pasos propuestos

1. **Agregar películas que disgustan.** Evaluar con Gemini 30–50 películas que la persona rechaza o le resultan indiferentes, eligiendo a propósito casos de cada filtro (en especial *Camaradería Masculina Rancia*). Amplía el rango de notas, da ejemplos positivos a los filtros y hace más informativo el ρ . Conviene descargar sus reseñas con el *scraper* antes de evaluarlas.
2. **Tres perfiles más.** Es la premisa del proyecto (multiperfil) y lo más barato de escalar: los embeddings de reseñas no dependen del perfil, así que un perfil nuevo solo necesita su JSON, la **opción 3** y la evaluación de Gemini de las mismas películas. Permite comprobar que la regla genérica se adapta a perfiles con otra cantidad de criterios y comparar la dificultad entre perfiles. Idealmente, perfiles con gustos contrastantes (p. ej. uno que valore el cine de acción o la comedia) para que las mismas películas tengan notas distintas.
3. **Evaluar las 57 candidatas** con la **opción 6** y descargar reseñas de las 9 películas evaluadas que no tienen: pasaría de 101 a unas 167 películas de entrenamiento. La curva de aprendizaje (fig. 7) indica cuánto puede rendir.
4. **Validar contra notas reales** de la persona (su diario de Letterboxd) y medir la consistencia de Gemini reevaluando unas 20 películas.
5. **Experimentos de modelo**, con ganancia más incierta: modelos de embeddings más grandes, como `multilingual-e5-large` o `bge-m3` (unas 5 h en Actions); entrenar la etapa 2 optimizando el orden en vez del error, y agregar frases de reseña a las afinidades.

A. Resumen de decisiones

Paso	Elegida	Descartada	Evidencia
Reseñas por película	100 con más <i>likes</i> , ≥ 100 caracteres	200 (igual o peor, el doble de lento); 50 (peor)	$\Delta\rho$ con MiniLM: 100 vs. 200, +0.021 (10/10)
Modelo de embeddings	e5-base (768 dim.)	MiniLM (básica), mpnet-base	ρ 0.674 vs. 0.605 y 0.651
Embedding del perfil	nombre + descripción	texto completo del criterio	similitud filtro–afinidad 0.68 $\rightarrow$ 0.50
Pooling	promedio	percentiles 50–95, promedio + similitudes	con e5 ninguna mejora ($\leq +0.001$)
Frases de reseña	ponderadas junto al embedding	concatenadas, solo frases, solo descripción	+0.008 en 10/10 y menor MAE
Modelo	reglas en dos etapas	regresión lineal (básica), Ridge	ρ 0.674 vs. 0.546 y 0.597
Etapas 2	regla simple (16 parámetros)	pisos y corrupción (31), GAM, <i>boosting</i> , <i>random forest</i>	MAE 0.294 vs. 0.293 con el doble de parámetros; árboles peores
Métrica	ρ de Spearman, 10 repeticiones	MAE, una partición	una partición varía ± 0.05

Tabla 9: Estrategias elegidas y descartadas en cada paso del pipeline.